



Tecnológico de Costa Rica

Curso:

Verificación funcional de circuitos integrados GR 1

Proyecto 2:

**Entorno de Verificación UVM para un Alineador de Bus de 32 bits con Interfaz
APB y RAL**

Profesor:

Ronny Giovanni Garcia Ramirez

Estudiantes:

Gael Agüero Carrillo
Kendy Raquel Arias Ortiz

Fecha:

Viernes 12 de Junio de 2026

1. Introducción

El presente proyecto consiste en el diseño e implementación de un entorno de verificación funcional completo para el módulo `cfs_aligner`, un alineador de bus de 32 bits con interfaz APB y soporte de interrupciones. El entorno fue desarrollado siguiendo la metodología UVM (Universal Verification Methodology), bajo un modelo estándar de verificación basado en transacciones.

El DUT recibe datos a través de una interfaz de memoria de datos (MD) de 32 bits organizada en 4 lanes, los alinea según la configuración establecida en sus registros de control, y los transmite por una interfaz de salida equivalente. El acceso a los registros internos del módulo se realiza mediante el protocolo APB (Advanced Peripheral Bus), y su abstracción en el entorno de verificación se implementa a través de un modelo RAL (Register Abstraction Layer) generado a partir de una descripción RDL.

El entorno integra cuatro agentes UVM (un agente RX activo, un agente APB activo, un monitor TX pasivo y un monitor IRQ pasivo) coordinados por un scoreboard central que verifica la corrección funcional del DUT, incluyendo la alineación de datos, el conteo de transferencias descartadas y el comportamiento del sistema de interrupciones.

2. Objetivos

Objetivo General:

Diseñar e implementar un entorno de verificación funcional UVM para el módulo `cfs_aligner`, que valide el comportamiento del alineador de bus de 32 bits, su interfaz APB y su sistema de interrupciones, mediante un modelo de verificación basado en transacciones con abstracción de registros mediante RAL.

Objetivos Específicos:

1. Construir el entorno de verificación UVM completo para el módulo `cfs_aligner`, incluyendo la interfaz SystemVerilog, los cuatro agentes requeridos, el scoreboard y su integración mediante puertos TLM.
2. Implementar el modelo RAL a partir de la descripción RDL de los registros del DUT, integrando el adaptador APB para permitir el acceso abstracto a los registros CTRL, STATUS, IRQEN e IRQ desde los tests.
3. Documentar el plan de pruebas y validar la ejecución del entorno mediante control de versiones en GitHub y simulación en un simulador compatible con SystemVerilog y UVM 1.2.
4. Verificar la correspondencia entre el comportamiento especificado y el comportamiento real del DUT, detectando y documentando discrepancias funcionales (tanto en el propio entorno de verificación como en el módulo bajo prueba) que permitan identificar defectos de diseño.

3. Arquitectura del Entorno

I. Estrategia de Solución y Desarrollo del Proyecto

El desarrollo del entorno de verificación se organiza en dos líneas de trabajo paralelas, que convergen en una arquitectura UVM unificada. Antes de comenzar la implementación se define en conjunto la estructura general del environment: los agentes necesarios (RX, APB, TX y monitor de IRQ), la interfaz

con el DUT, el modelo de registros a utilizar y el esquema de verificación basado en un scoreboard con modelo de referencia. Esta definición temprana permitió que ambas líneas de trabajo avanzaran sin generar incompatibilidades mayores al momento de la integración.

La primera línea de trabajo se enfoca en la capa de acceso a registros y se basa en el uso de las bibliotecas de RAL para generar un modelo del DUT a nivel de registros que permita un fácil manejo desde el ambiente UVM, se usa PeakRDL para generar este modelo RAL.

La segunda línea de trabajo se centra en el environment de verificación propiamente dicho: la interfaz SystemVerilog (`aligner_if.sv`), el paquete del testbench (`aligner_tb_pkg.sv`), el environment (`aligner_en.sv`), los agentes de la interfaz MD (driver RX y monitores TX/IRQ: `md_components.sv`), junto con las secuencias del estímulo (`md_sequences.sv`), y un primer conjunto de tests generales sobre el módulo `cfs_aligner` y `dut.sv`.

Una vez completadas ambas partes, se integran en un único repositorio bajo el control de versiones de GitHub, y se realizan ajustes para asegurar compatibilidad entre componentes (adecuación de las transacciones y puertos TLM entre los agentes propios y el modelo RAL incorporado, conexión del adaptador APB al driver correspondiente, mejora del scoreboard para integrar correctamente las nuevas vías de configuración del DUT (vía RAL) con el modelo de referencia. A partir de esta integración, el desarrollo continúa de forma conjunta, se depuran los tests, se ajusta el scoreboard y la corrección de bugs detectados se realiza sobre la base ya unificada mediante ciclos iterativos de simulación, análisis de logs y resultados.

II. Descripción de los Módulos del Entorno

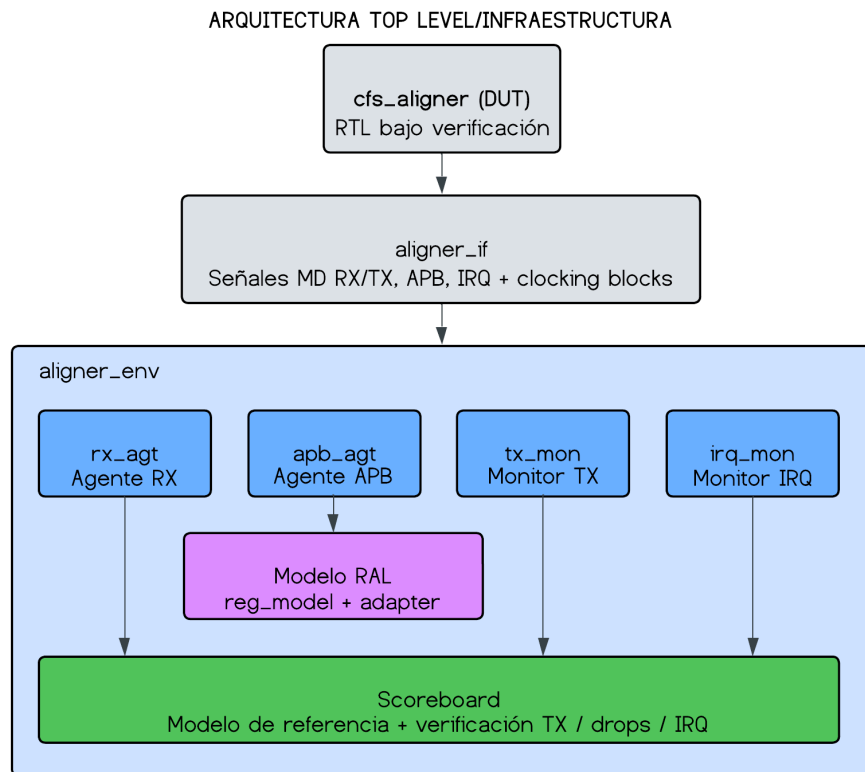


Figura 1. Diagrama de la arquitectura del environment con los bloques agentes, scoreboard, RAL/adapter y sus conexiones TLM.

a. Top-level / infraestructura

- tb_top.sv

El módulo `tb_top.sv` actúa como el módulo top de la simulación, y el componente que mantiene unificado todo el entorno físicamente, siendo el encargado de instanciar, conectar y configurar todos los elementos del banco de pruebas. Para ello incluye los archivos del DUT, el RAL, los componentes APB y MD, junto con el scoreboard, integrándolos mediante el `import` de sus respectivos paquetes. En su interior, el módulo declara las señales `clock` y `reset`, e instancia la interfaz `aligner_if` como la vía explícita de conexión entre el testbench y el diseño.

De la misma manera, realiza la instanciación del DUT (`cfs_aligner`) configurando los parámetros: `ALGN_DATA_WIDTH=32` y `FIFO_DEPTH=8`, mapeando sus puertos de MD (RX/TX), APB e IRQ a las señales de la interfaz virtual (`vif`). A nivel de control de señales genera un reloj de 50 MHz con un período de 20 ns, y un ciclo de reset inicial que se libera tras 10 ciclos. Finalmente, para habilitar la infraestructura de UVM, el módulo publica en el `uvm_config_db` los modports específicos de la interfaz destinados a cada agente y monitor (`rx_driver_mp`, `rx_monitor_mp`, `tx_driver_mp`, `tx_monitor_mp`, `irq_mp` y la interfaz APB completa), crea y registra el modelo RAL (`cfs_aligner_regs`), y ejecuta el método `run_test()` para ceder el control al environment e iniciar formalmente la simulación.

- aligner_if.sv

El módulo `aligner_if.sv` es la interfaz SystemVerilog que define el contrato de señales físicas del DUT, organizándolas en cuatro grupos principales: MD RX, MD TX, APB e IRQ. Con el fin de evitar carreras de simulación, la interfaz incorpora clocking blocks (`rx_driver_cb`, `rx_monitor_cb`, `tx_driver_cb`, `tx_monitor_cb`, `apb_cb` e `irq_cb`) que sincronizan el acceso a las señales con el flanco de reloj, delimitando estrictamente cuáles actúan como entradas o salidas según el rol de driver o monitor.

Asimismo, implementa modports para ofrecer vistas restringidas y direccionales de estas señales a cada componente del testbench (`rx_driver_mp`, `rx_monitor_mp`, `tx_driver_mp`, `tx_monitor_mp`, `apb_driver_mp`, `apb_monitor_mp` e `irq_mp`), incluyendo un modport específico para el propio DUT. Finalmente, la interfaz añade una capa de verificación de protocolo a nivel de señal mediante aserciones estáticas e independientes del scoreboard; entre ellas destacan `ast_rx_hold`, que valida la estabilidad de los datos, `offset` y `size` cuando `md_rx_valid` está activo y `md_rx_ready` es bajo, y `ast_rx_size`, que asegura que nunca se transmita un paquete RX con un tamaño igual a cero.

- aligner_tb_pkg.sv

El módulo `aligner_tb_pkg.sv` es el paquete encargado de estructurar el "vocabulario" común del testbench mediante la definición de las transacciones u objetos UVM que fluyen entre los agentes y el scoreboard.

En primer lugar, modela la clase `rx_transaction` para representar los paquetes de entrada (MD RX), conteniendo los campos de datos, `offset`, `size`, `valid` y `err`; este objeto incorpora constraints de aleatoriedad esenciales como `c_size_not_zero` (restringiendo el tamaño entre 1 y 4), `c_legal` (que fuerza la regla matemática de legalidad del protocolo), y diversas distribuciones probabilísticas para guiar las combinaciones de estímulos. A partir de esta, se deriva `rx_transaction_illegal`, una clase heredada que

invierte la restricción de legalidad mediante `c_force_illegal` para generar deliberadamente paquetes erróneos que el DUT debe descartar, permitiendo así la verificación de las rutas de error. Por otro lado, el paquete define `tx_transaction` para los paquetes de salida (MD TX) (el cual fuerza el offset a cero y restringe el size según la especificación) e `irq_transaction`, que registra las detecciones de interrupción junto con su respectivo timestamp. Finalmente, la infraestructura del paquete incluye las macros de declaración `uvm_analysis_imp_decl` para los canales de recepción (`_rx`, `_tx` e `_irq`), generando los puertos de análisis específicos que el scoreboard requiere para procesar cada flujo de datos de manera aislada y organizada.

- **aligner_env.sv**

El módulo `aligner_env.sv` define la clase `aligner_env`, el contenedor estructural (`uvm_env`) encargado de instanciar y conectar todos los componentes del entorno mediante transacciones TLM.

Durante su fase de construcción (`build_phase`), el environment se encarga de crear el agente APB (`apb_agt`), el agente RX (`rx_agt`), el monitor de TX (`tx_mon`) junto con su respectivo driver de ready (`tx_rdy_drv`), el monitor de IRQ (`irq_mon`) y el scoreboard (`sb`); asimismo, recupera el modelo RAL (`reg_model`) desde el `uvm_config_db` para construir el adaptador APB (`adapter`) y el predictor de registros (`uvm_reg_predictor`). Posteriormente, en la fase de conexión (`connect_phase`), se materializa la arquitectura de verificación al enlazar los puertos de análisis de los monitores con los accesos independientes del scoreboard (`rx_agt.ap`, `tx_mon.ap` e `irq_mon.ap` hacia sus respectivos exports). En este mismo bloque de conexiones, el mapa por defecto del RAL (`default_map`) se acopla al sequencer del agente APB mediante el adaptador, configurando la predicción explícita con `set_auto_predict(0)`, mientras que el monitor APB se conecta a la entrada del predictor (`bus_in`) para asegurar que el modelo refleje en tiempo real el estado del DUT antes de fijar la estructura mediante `reg_model.lock_model()`. Por último, la clase expone hacia los tests los métodos abstractos `set_sb_config()`, empleado para notificar cambios de configuración en el modelo de referencia, y `verify_drops()`, utilizado para contrastar el contador de paquetes descartados frente a los registros leídos por el bus APB.

b. Modelo de Registros (RAL)

- **cfs_aligner.rdl**

Este módulo es el modelado en RDL que usamos para generar el equivalente en UVM con PeakRDL que se escribe en el estándar SystemRDL.

- **cfs_aligner_ral_pkg.sv**

Este es el módulo en SystemVerilog producido por PeakRDL que actúa como el modelo RDL de los registros del sistema.

c. Agentes

- **apb_componentes.sv**

El módulo `apb_componentes.sv` implementa la clase `apb_agent` para centralizar la comunicación con el bus APB del DUT a través de tres subcomponentes y un adaptador dedicados.

La estructura se compone de `apb_transactions`, el ítem de secuencia que modela las transferencias (dirección de 16 bits, datos de 32 bits, control de escritura y error) restringiendo la aleatoriedad a las direcciones reales de los registros (CTRL, STATUS, IRQEN, IRQ); el `apb_driver`, encargado de ejecutar las fases del protocolo en el bus y el `apb_monitor` que observa pasivamente estas señales para publicar cada transacción completada mediante su `analysis_port` hacia el scoreboard. Finalmente, el agente incorpora el `apb_reg_adapter` como puente con el RAL, que traduce las operaciones lógicas de registros a transacciones físicas mediante `reg2bus()`, y decodifica las respuestas del bus hacia el modelo con `bus2reg()`, mapeando la señal `pslverr` al estado de UVM.

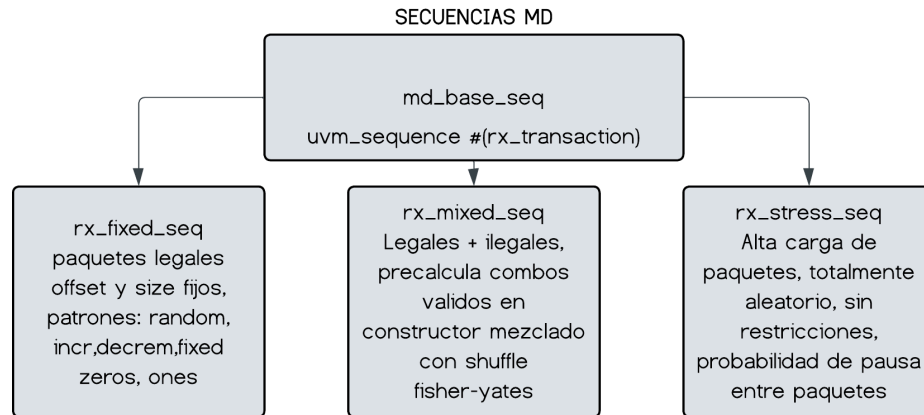


Figura 2. Diagrama de herencia y jerarquía de las Secuencias MD, se pueden observar los distintos tipos de estímulos de prueba.

- `md_componentes.sv`

Implementa la infraestructura de datos del entorno mediante el agente activo de recepción (`rx_agent`), dos monitores pasivos y un controlador de contrapresión, aislando las vistas mediante `modports`.

El `rx_driver` inyecta transacciones considerando el backpressure del DUT al esperar la señal `md_rc_ready` antes de finalizar el ítem o aplicando un estado pasivo si el paquete no es válido. Por otra parte el `rx_monitor`, captura los handshakes de entrada recalculando el error combinacional de forma local, esto para evitar problemas de timing, mientras que el `tx_monitor` observa pasivamente la salida del DUT para publicar las transferencias completadas hacia el scoreboard. Finalmente el `tx_ready_driver` controla de manera autónoma la señal `md_tx_ready` alternando entre un estado de siempre listo o aleatorio mediante el flag `ready_always_one`, en tanto el `irq_monitor` registre y emite marcas de tiempo ante cada flanco positivo de la interrupción del sistema.

- `md_sequences.sv`

Define los flujos de estímulos del entorno a través de tres clases especializadas que heredan de `md_base_seq`. La secuencia funcional básica, genera una cantidad fija de paquetes legales con dimensiones estáticas y soporte para múltiples patrones de datos (Random, Incremental, Decremental, Fixed Zeros, Ones). Como contraparte avanzada, precalcula en su constructor todas las combinaciones especiales válidas y utiliza el algoritmo Fisher-Yates shuffle para intercalar aleatoriamente paquetes

legales con transacciones inválidas. Por último, ejecuta cargas masivas de transacciones completamente aleatorias introduciendo retardos probabilísticos entre transferencias, permitiendo evaluar la solidez del diseño ante saturación y huecos de tráfico de bus.

d. Verificación

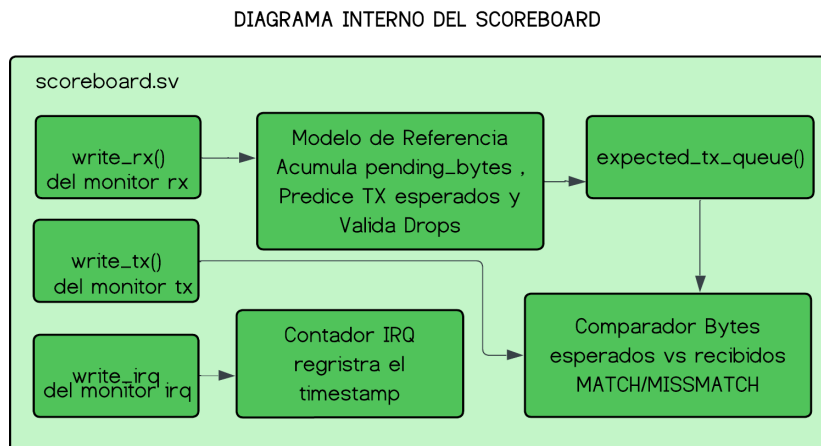


Figura 3. Diagrama del flujo general interno del scoreboard donde se ilustra el proceso de comparar la salida del DUT con el Modelo de Referencia para validar los datos.

- scoreboard.sv

El scoreboard tiene la lógica central de verificación comparando en tiempo real el comportamiento del hardware contra las predicciones de un modelo de referencia (`ref_model`). Este modelo en software simula el pipeline del alineador acumulando los bytes válidos de las transacciones RX en una cola interna para estructurar los paquetes de salida proyectados dentro de la lista `expected_tx_queue`, descartando automáticamente los estímulos que incumplan las reglas de legalidad del protocolo. Por su parte, el scoreboard actúa como un árbitro clasificando los flujos de entrada, procesando los eventos de interrupción y contrastando las transacciones capturadas por el monitor de salida contra los elementos esperados; esta comparación se ejecuta de manera deliberada únicamente sobre los bytes activos para tolerar las desalineaciones de offset remanentes del diseño. Adicionalmente, el entorno introduce el flag `armed` como un mecanismo de control que se deshabilita durante los ciclos de reset para ignorar falsos positivos antes de que las secuencias tomen control efectivo. Al concluir la simulación, la fase de chequeo, audita los contadores globales y discrimina si los paquetes pendientes en las colas corresponden a palabras incompletas tolerables o omisiones de transmisión por parte del hardware.

e. Tests y Regresión

- test_general.sv

Implementa la clase `test_general`, la cual es completamente configurable en tiempo de ejecución mediante plusargs (`TEST_MODE`, `CTRL_SIZE`, `CTRL_OFFSET`, `MD_NUM_PKTS`, `MD_PESO_LEGAL` y la semilla de randomización), evitando la necesidad de recompilar el entorno entre escenarios.

Durante su fase de ejecución (run_phase), el componente realiza la programación inicial de los registros CTRL e IRQEN a través del RAL, validando la legalidad de los parámetros ingresados. Seguidamente, ejecuta una rutina de limpieza en el scoreboard para ignorar de manera segura cualquier residuo transitorio en el hardware antes de habilitar el arbitraje activo. Dependiendo del modo seleccionado, el test conmuta dinámicamente entre tres flujos operativos: APB_ONLY, que inyecta transferencias aleatorias sobre el bus de control y gestiona las interrupciones; MD_ONLY, que ejecuta la secuencia mixta de paquetes de datos controlando el balance de legalidad; o FULL, que ejecuta ambos flujos concurrentemente mediante un bloque fork/join desfasado en tiempo. Finalmente, la lógica realiza una auditoría pos-simulación leyendo el registro STATUS para contrastar los contadores de descarte físicos mediante el método verify_drops() del environment, cediendo el veredicto definitivo a la fase de chequeo de la infraestructura UVM.

- **aligner_regression.sh**

Este es el script usado para correr las simulaciones, es el código con el que el usuario interactúa. Hace uso de los plusargs para aleatorizar todas las variables que necesite aleatorizar, hace una recopilación del UVM en forma de logs, hace un resuy simen de cada corrida y de la configuración que tiene cada corrida al igual que decir si la prueba pasó o falló.

f. DUT

- **dut.sv**

Representa el núcleo de hardware bajo verificación, actuando como un alineador de datos de 32 bits que recibe paquetes en formatos arbitrarios para reorganizarlos según la configuración de sus registros internos. Su arquitectura física se divide en cuatro bloques lógicos esenciales e instanciados bajo un único dominio de reloj. El módulo de recepción evalúa de forma combinatorial la legalidad del protocolo en cada paquete entrante, activando una señal de error y registrando el descarte si la combinación es inválida, o almacenando los datos válidos en una cola interna mientras gestiona la contrapresión hacia la interfaz de entrada. El bloque de procesamiento central actúa como el motor del alineador, extrayendo los bytes útiles, acumulándolos hasta completar el tamaño configurado y estructurando las nuevas palabras de salida para transferirlas a la cola de transmisión. El bloque de salida expone de manera directa estos paquetes procesados hacia la interfaz externa, controlando el flujo mediante el saludo de validación y aceptación con el receptor. Por último, el bloque de control opera como esclavo del bus de periféricos avanzado, gestionando el acceso a los registros de configuración, estado, habilitación y banderas de interrupción de tipo persistente, validando en tiempo real que las escrituras del bus cumplan con las restricciones de diseño.

3. Plan de Pruebas y Estrategia de Validación

La ejecución del entorno de verificación se realizó sobre los servidores de la universidad, que disponen del simulador VCS de Synopsys con soporte para SystemVerilog y UVM 1.2. El acceso se realiza por terminal mediante conexión remota, cargando el entorno de herramientas con el script institucional synopsys_tools2.sh antes de cada sesión de simulación. Toda la compilación y ejecución de pruebas se ejecuta desde línea de comandos, sin interfaz gráfica.

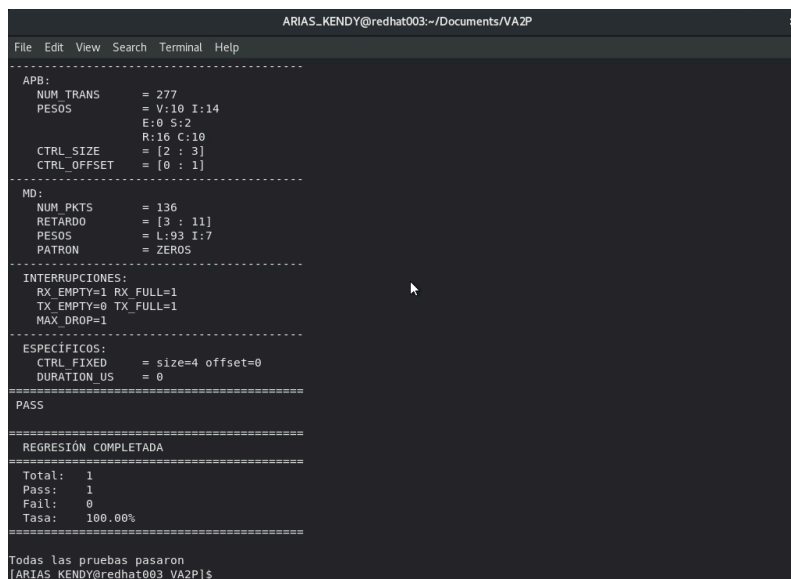
La validación del entorno se realizó mediante el script `aligner_regression.sh`, que automatiza la compilación y ejecución de una batería de pruebas aleatorias. Se invoca desde terminal especificando el número de pruebas a correr:

```
./aligner_regression.sh [NUM_TESTS]
```

El script compila el testbench una única vez con VCS antes de ejecutar los tests, lo que garantiza que todos corren contra el mismo binario. Para cada prueba genera aleatoriamente un conjunto completo de parámetros independientes:

1. **Semilla:** generada combinando dos llamadas a `$RANDOM` de bash, lo que produce valores de hasta `~1 073 741 823`. Es el parámetro clave de reproducibilidad: con la misma semilla y los mismos parámetros se obtiene exactamente la misma secuencia de estímulos.
2. **Modo de test:** con distribución ponderada 60% FULL (APB + MD simultáneos), 20% APB_ONLY, 20% MD_ONLY.
3. **Parámetros APB:** número de transacciones (50–500), pesos relativos por tipo de acceso (CTRL válido/inválido, IRQEN, STATUS, IRQ, IRQ clear) y rangos de size/offset para el registro CTRL.
4. **Parámetros MD:** número de paquetes (20–220), retardos entre transacciones, proporción de paquetes legales vs ilegales, y patrón de datos (RANDOM, INCR, DECR, FIXED, ZEROS, ONES).
5. **Interrupciones:** habilitación individual aleatoria de cada fuente de IRQ (`rx_empty`, `rx_full`, `tx_empty`, `tx_full`, `max_drop`).

Si una prueba falla, el script la reintenta hasta dos veces con una semilla modificada (SEMILLA + intento) antes de registrarla como fallo definitivo. Los tests que pasan guardan su configuración en `logs/passed_configs.txt`; los que fallan guardan la configuración en `failed.txt` y preservan el log completo en `logs/failed_test_N.log` para análisis posterior. Al finalizar, el script imprime un resumen con totales de pass/fail y tasa de éxito.



```
ARIAS_KENDY@redhat003:~/Documents/VA2P
File Edit View Search Terminal Help
-----
APB:
NUM_TRANS      = 277
PESOS          = V:10 I:14
               E:0 S:2
               R:10 C:10
CTRL_SIZE      = [2 : 3]
CTRL_OFFSET    = [0 : 1]
-----
MD:
NUM_PKTS       = 136
RETARDO        = [3 : 11]
PESOS          = L:93 I:7
PATRON         = ZEROS
-----
INTERRUPCIONES:
RX_EMPTY=1 RX_FULL=1
TX_EMPTY=0 TX_FULL=1
MAX_DROP=1
-----
ESPECIFICOS:
CTRL_FIXED     = size=4 offset=0
DURATION_US    = 0
=====
PASS
=====
REGRESIÓN COMPLETADA
=====
Total:  1
Pass:   1
Fail:   0
Tasa:   100.00%
=====
Todas las pruebas pasaron
[ARIAS_KENDY@redhat003 VA2P]$
```

Figura 4. Ejemplo de una regresión completada con éxito desde la terminal del servidor de la escuela de Electrónica.

3.1 Defecto identificado en el DUT

Durante la ejecución de las regresiones continuas y debugging, se identifica un defecto funcional en el módulo `cfs_aligner` (DUT) que se manifiesta de forma consistente en un subconjunto de las pruebas ejecutadas. El scoreboard del entorno lo detecta y reporta como TX MISMATCH, lo cual constituye el comportamiento correcto del entorno de verificación ya que su función es precisamente detectar y reportar discrepancias entre el comportamiento real del DUT y el comportamiento esperado según la especificación.

Cuando el DUT recibe un paquete RX con `size > 1` (es decir, un paquete que requiere generar múltiples palabras TX de salida), los primeros TX de la ráfaga se producen correctamente, pero el último TX de acumulación (el que corresponde al byte restante cuando no hay suficientes bytes para completar un word) sale con el campo de datos en `0x00000000` en lugar del valor esperado. El defecto se localiza en el módulo `cfs_ctrl` del DUT, en las ramas de acumulación de `push_data`, donde se realiza una asignación completa de 32 bits que zero- extiende involuntariamente los bits de metadatos [36:32] del FIFO interno.

El defecto es completamente reproducible porque se activa únicamente cuando concurren las condiciones `size_rx > ctrl_size` con acumulación de bytes residuales, y se puede forzar de forma determinista pasando la semilla y los parámetros exactos de cualquier prueba fallida registrada en `failed.txt`. Por ejemplo, la prueba con semilla `106267324` reproduce el defecto con `TEST_MODE=MD_ONLY`, `MD_NUM_PKTS=96` y `MD_PATRON=FIXED`, generando 8 errores del tipo descrito. En contraste, pruebas con las mismas condiciones generales pero semillas que no generan el escenario de acumulación residual (como las registradas en `passed_configs.txt`) pasan sin errores, lo que confirma que el entorno de verificación es funcionalmente correcto y que los fallos reportados corresponden al comportamiento defectuoso del DUT.

Para reproducir cualquier prueba fallida de forma exacta se ejecuta directamente el binario de simulación con los parámetros del registro correspondiente en `failed.txt`:

1. Cargar el entorno de Synopsys

```
source /mnt/vol_NFS_rh003/estudiantes/archivos_config/synopsys_tools2.sh
```

2. Compilar

```
vcs -sverilog -timescale=1ns/1ps -ntb_opts uvm-1.2 tb_top.sv -o simv -l compile.log
```

3. Ejecutar con los parámetros de la prueba fallida

```
./simv +SEMILLA=106267324 +TEST_MODE=MD_ONLY +MD_NUM_PKTS=96 \
+MD_PATRON=FIXED +UVM_TESTNAME=test_general
```

```
=====
TEST GENERAL CONFIGURATION
=====
SEMILLA      = 106267324
TEST_MODE    = MD_ONLY
APB_NUM_TRANS = 100
MD_NUM_PKTS  = 96
MD_PESO_LEGAL = 80%
CTRL_SIZE    = 2
CTRL_OFFSET  = 0
MD_PATRON    = FIXED
TIMEOUT_MS   = 100
=====
```

Figura 5. Configuración del test para usar una semilla que causaría problema debido al bug del DUT.

```

UVM_INFO scoreboard.sv(213) @ 1530000: uvm_test_top.env.sb [scoreboard] [TX] data=0x0000a5a5 off=0 size=2
UVM_INFO scoreboard.sv(252) @ 1530000: uvm_test_top.env.sb [scoreboard] TX MATCH OK: bytes=2 data_exp=0x0000a5a5
data_rec=0x0000a5a5
UVM_INFO scoreboard.sv(213) @ 1550000: uvm_test_top.env.sb [scoreboard] [TX] data=0x000000a5 off=0 size=2
UVM_ERROR scoreboard.sv(245) @ 1550000: uvm_test_top.env.sb [scoreboard] TX MISMATCH:
  Esperado: 0x0000a5a5 (off=0 size=2)
  Recibido: 0x000000a5 (off=0 size=2)

```

Figura 6. Mensaje de ERROR UVM por un mismatch generado por un bug en el DUT.

```

=====
RESUMEN SCOREBOARD
=====
RX packets recibidos: 96
Drops esperados: 20
Drops reales (CNT DROP): 20
TX generados (modelo): 77
TX recibidos: 77
TX correctos: 69
TX incorrectos: 8
IRQs recibidas: 89
Errores totales: 8
=====

```

Figura 7. Resultado final de la semilla problemática con 8 errores totales de mismatch por bug del DUT.

La corrección de este defecto corresponde al diseñador del RTL, ya que implica modificar `dut.sv`, que está fuera del alcance de este proyecto de verificación. El entorno queda documentado como funcional, detecta el defecto de forma consistente y reproducible, que es el resultado esperado de un entorno de verificación ante un DUT con errores de implementación.

El código fuente completo del entorno de verificación desarrollado en este proyecto se encuentra disponible en el siguiente repositorio:

<https://github.com/Gael-Aguero/Proyecto-2-Verificacion-funcional.git>

4. Conclusiones

1. El proyecto permitió construir un entorno de verificación UVM completo para el módulo `cfs_aligner`, aplicando en la práctica los conceptos de generación de estímulos, abstracción de registros mediante RAL y verificación automática con scoreboard.
2. Se detecta un defecto en el DUT durante la regresión. El scoreboard reportó de forma consistente y reproducible un TX MISMATCH en el byte de acumulación residual de ráfagas multi-byte, lo que confirma que el entorno cumple su función (detectar comportamientos incorrectos del hardware, no simplemente ejecutar pruebas). Un entorno que reporta fallos ante un DUT defectuoso no está fallando, está funcionando correctamente.
3. El uso de semillas como mecanismo de reproducibilidad resultó importante para el debugging y la documentación, permitiendo aislar el defecto con precisión y construir evidencia objetiva a partir de parámetros registrados en `failed.txt`.

5. Bibliografía

- Material del curso Verificación Funcional de Circuitos Integrados, Tecnológico de Costa Rica, 2026.
- Apuntes y ejemplos proporcionados por el profesor durante el desarrollo del curso.

- IEEE. (2017). *IEEE Standard for SystemVerilog - Unified Hardware Design, Specification, and Verification Language*. IEEE Std 1800-2017.
- Accellera Systems Initiative. (2015). *Universal Verification Methodology (UVM) 1.2 User's Guide*. Accellera.
- Real, F. (2016). *SystemVerilog for Verification: A Guide to Learning the Testbench Language Features* (3rd ed.). Springer.
- SystemRDL Consortium. (2018). *SystemRDL 2.0 Specification*. Accellera Systems Initiative.
- PeakRDL Documentation. Recuperado de <https://peakrdl.readthedocs.io>